Two Independent LEDs

Pins 4 and 7 on the Arduino Mega 2560

Build deliberately, predict first, and support every claim with repeated evidence.

Lesson	002	**Time**	70–100 minutes
Level	First external-output circuit	**Board**	Arduino Mega 2560
Outcome	Independently command and verify two current-limited LEDs	**Evidence**	Prediction, measurements, three trials, CER

1 Why this circuit matters

The built-in LED hides its wiring. Here, you make every electrical choice: output pin, current-limiting resistor, LED polarity, and return path. Two branches then reveal an important idea: components can share ground and still be controlled independently. That pattern expands naturally to status panels, traffic signals, displays, and diagnostic indicators.

Learning targets

By the end, you can:

- trace each complete path from a Mega output pin to ground;
- identify an LED's anode and cathode before energizing;
- calculate a safe resistor value and estimate branch current;
- distinguish a wiring observation from a software claim;
- collect repeated evidence that pins 4 and 7 control separate branches;
- explain how an RAII component can leave an output in a safe state.

2 Roles and safety

Work alone or use two roles. Swap roles after the first successful trial.

Builder / operator
Keeps all power sources disconnected while changing wiring, reads resistor bands, places parts, and announces before power is applied.

Verifier / recorder
Traces the exact schematic, checks LED polarity and shared ground, records predictions and measurements, and calls "stop" if a connection differs from the schematic.

Safety checkpoint. Disconnect *all* power sources—USB, barrel-jack supply, battery, programmer, and powered external modules—before inserting, moving, or removing any wire or component. Never connect an LED directly between an output and ground: each LED needs its own series resistor. Do not connect pins 4 and 7 together. If a component becomes warm, an LED is unexpectedly bright, or the board resets, disconnect power immediately.

Required: Mega 2560, USB cable, breadboard, two LEDs (preferably visibly different colors), two 330 Ω resistors, jumper wires, and a multimeter. Use the 20 V DC range (or auto-range) for voltage. Current measurement is optional and must be done only with an instructor-approved series connection; never place a meter in current mode directly across a pin and ground.

3 Predict before building

The sketch alternates the LEDs: red on/blue off, then red off/blue on. Complete these predictions before touching the hardware.

Pin 4 level	Pin 7 level	Predicted visible result	Why?
HIGH	LOW	_____	_____
LOW	HIGH	_____	_____
LOW	LOW	_____	_____

De-energized-state prediction: With all power disconnected, what should each LED do? _____
Why can an LED not remain lit from these pin branches? _____

Fault prediction: If the blue LED is reversed but every other connection is correct, predict what red and blue will do: _____

4 Two drawings, two jobs

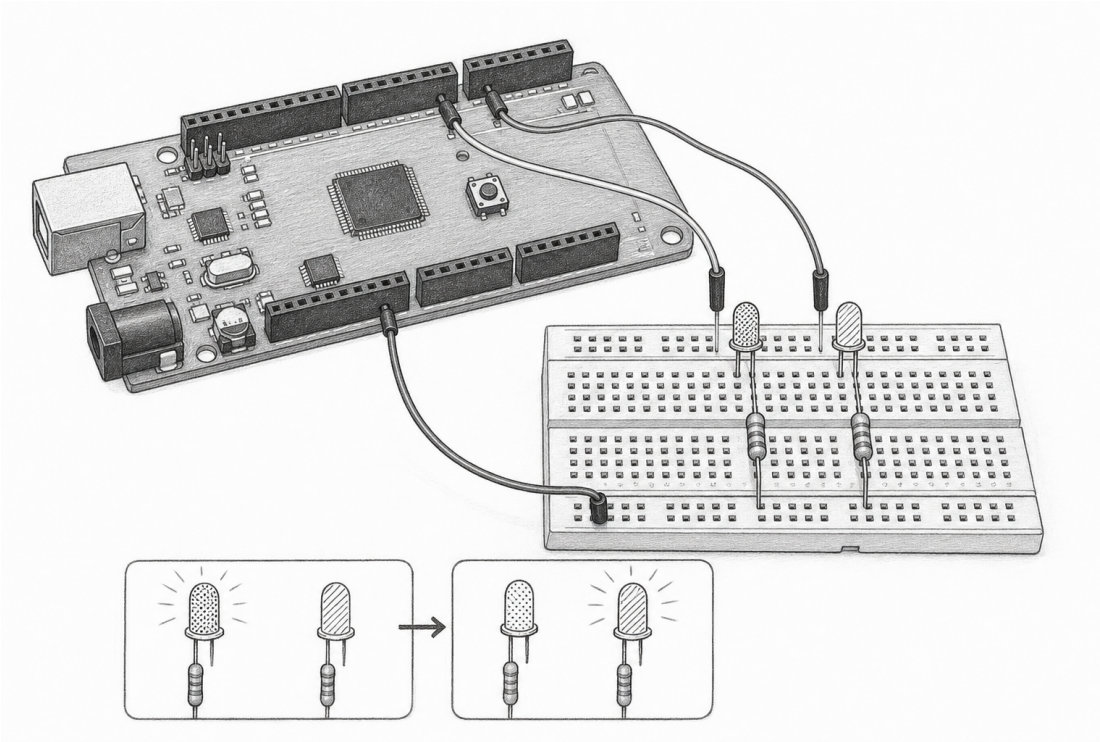
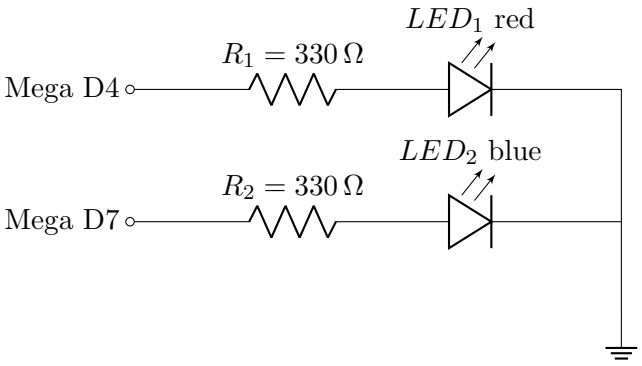


Figure 1: Conceptual orientation only. This pencil drawing communicates the idea of two LEDs, but it is **not literal wiring guidance**. It may simplify breadboard rows, pin positions, polarity, or resistor placement. Build from the exact schematic and connection table below.

4.1 Exact electrical schematic



Two separate branches
Each branch has its own resistor.
Both cathodes return to Mega GND.

Conventional current flows from a HIGH output, through that branch's resistor, through the LED from anode to cathode, and back through GND. On a common through-hole LED, the longer lead is usually the anode; the flat edge and shorter lead usually mark the cathode. Verify the part itself because leads may have been trimmed.

From	Through	To	Verification
Mega D4	$R_1 = 330\,\Omega$, then red LED anode	red cathode to GND	branch 1 only
Mega D7	$R_2 = 330\,\Omega$, then blue LED anode	blue cathode to GND	branch 2 only

Important breadboard rule: The resistor and LED must be in series, not merely near each other. Use a continuity check with power disconnected if the breadboard's connected rows are unfamiliar.

5 Choose the resistor with a calculation

The resistor absorbs the voltage not dropped by the LED. Estimate branch current:

$$I = \frac{V_{\text{pin}} - V_f}{R}.$$

Using an approximate 5 V HIGH and 330 Ω:

$$I_{\text{red}} \approx \frac{5\,\text{V} - 2.0\,\text{V}}{330\,\Omega} = 9.1\,\text{mA}, \qquad I_{\text{blue}} \approx \frac{5\,\text{V} - 3.0\,\text{V}}{330\,\Omega} = 6.1\,\text{mA}.$$

These are pre-power estimates: actual pin HIGH voltage and LED forward voltage vary. A 330 Ω resistor is deliberately conservative and normally provides a clearly visible result without chasing maximum brightness.

Pre-power calculation for your parts: Use a datasheet value or reasonable color-based estimate for V_f. Do not try to measure V_f until the inspected, current-limited circuit is safely energized.

Branch	Assumed V_{pin}	Assumed LED V_f	R	Predicted I
D4 / LED 1	_____	_____	_____	_____
D7 / LED 2	_____	_____	_____	_____

6 De-energized build procedure

1. **Disconnect all power.** Confirm: ☐ USB disconnected; ☐ no barrel/battery/programmer/module power; ☐ board power LED off.
2. Identify D4, D7, and a GND pin on the printed Mega labels. Do not infer positions from the pencil drawing.
3. Read both resistor color codes and verify approximately 330 Ω with the meter if available.
4. Build the D4 branch exactly as the schematic shows.
5. Build the D7 branch exactly as the schematic shows.
6. Trace aloud: "D4, resistor, LED anode-to-cathode, ground," then repeat for D7.
7. Check there is no wire joining D4 to D7 and no branch bypassing a resistor.
8. Ask the verifier to initial the independent inspection: _____.

> **Power-on gate:** Do not connect any power source until every item above is checked. A circuit that is unpowered is the correct state for inspection, continuity tests, and changes.

7 Software: alternate the two outputs

The current repository example for this lesson expresses each LED as a component:

```
#include <adk.h>

adk::led::Mono red  (4);
adk::led::Mono blue (7);

void setup()
{
    adk::initialize();
}

void loop()
{
    // A one-second dwell gives a DMM time to settle.
    constexpr auto intervalMs = 1000;

    red .on  ();
    blue.off ();
    delay(intervalMs);

    red .off ();
    blue.on  ();
    delay(intervalMs);
}
```

For this measurement exercise, set the supplied example's interval to 1000 ms before uploading. Related member calls align their parentheses so state transitions can be compared quickly.

7.1 RAII and the component boundary

The sketch should say *what* the red and blue components do, while the library owns *how* a pin is configured and returned to a safe state. The intended component contract uses a `struct`, normalized CamelCase names, out-of-line implementation, explicit fallible initialization, and automatic cleanup:

```
struct Mono
{
    explicit Mono (PinId pin) noexcept;
             ~Mono() noexcept;

    Result initialize  () noexcept;
    void   shutdown    () noexcept;
    bool   initialized () const noexcept;

    void   on          () noexcept;
    void   off         () noexcept;
};
```

> **Safety checkpoint. Planned API, not current executable API:** the lifecycle interface above is a design target and is not yet implemented by this lesson's library. Do not copy it into the sketch expecting it to compile. The executable example currently uses `adk::initialize()` and the available `Mono::on()/off()` calls.

`begin()` and `end()` are avoided because those names conventionally describe iterator boundaries. Internally, ADK does not throw. Nevertheless, RAII means that if application code is compiled with exceptions and stack unwinding occurs, ∼`Mono()` still calls an idempotent `shutdown()`. For an LED, shutdown first commands the inactive level; a generic output can then use a high-impedance policy. Constructors should establish an inert valid object; `initialize()` should claim/configure hardware transactionally and report failure as a `Result`. Implementation stays in a `.cpp` wherever templates or compile-time evaluation do not require header definitions.

Design prompt: Why is a `Mono` best modeled as owning its output-pin resource rather than borrowing an untracked global pin? _____

8 Observe, measure, repeat

Connect USB only after the power-on gate. Build and upload the lesson with the project's Arduino CLI wrapper, or open the supplied `lessons/002/002.ino` using a Mega 2560 board selection. Confirm that its measurement dwell is 1000 ms, then observe at least three complete red/blue cycles.

8.1 Voltage measurements

Keep the black probe on Mega GND. Touch the red probe to D4, then D7. Record a representative HIGH and LOW. Do not let a probe bridge adjacent header pins. Only after those measurements, measure LED forward voltage by placing probes across a lit LED: red probe at the anode and black probe at the cathode.

Quantity	Trial 1	Trial 2	Trial 3	Pattern / notes
D4 when red is on (V)				
D4 when red is off (V)				
D7 when blue is on (V)				
D7 when blue is off (V)				
Red LED V_f, while lit (V)				
Blue LED V_f, while lit (V)				

8.2 Repeated behavioral evidence

Cycle	D4 command	Red observed	D7 command	Blue observed	Cross-control or anomaly?
1	ON/OFF		OFF/ON		
2	ON/OFF		OFF/ON		
3	ON/OFF		OFF/ON		

Aggregate-current boundary: The two branches would draw approximately $9.1\,\text{mA} + 6.1\,\text{mA} = 15.2\,\text{mA}$ if both were on. The base alternation draws only one branch at a time. Do not infer that any number of such branches may be added: pin, port-group, and whole-MCU supply/ground limits all apply simultaneously. Recalculate the worst-case sum, check the ATmega2560 and board limits, and use an external driver and suitable power supply before scaling to many loads.

De-energized-state observation: Disconnect all power after the trials. Record the visible state of both LEDs and compare it with the earlier prediction.

---

9 Troubleshooting by symptom

Disconnect all power *before* applying any wiring remedy.

Symptom	Likely cause	De-energized check or controlled test
One LED never lights	LED reversed, open row, wrong pin, or damaged LED	Check cathode/anode, trace that branch, continuity-test rows, then swap only the LED with a known-good one.
Both LEDs always match	Pins or branches accidentally joined, or code addresses one pin twice	Verify D4 and D7 are not connected; compare constructor pin numbers.
LED is extremely bright or board resets	Missing/bypassed resistor or short circuit	Disconnect immediately; verify one 330 Ω resistor is in series in each branch.
Sequence is reversed	LED colors/branches were exchanged or prediction mapped pins incorrectly	Trace from D4 and D7 rather than assuming physical left/right position.
Flicker or intermittent light	Loose lead, split breadboard rail, or poor ground	Gently reseat with power off; continuity-test the common ground path.
Upload succeeds but neither lights	Missing ground, both LEDs reversed, wrong board/port, or initialization failure	Confirm shared GND and polarity; verify Mega 2560 selection; inspect returned initialization status when the interface exposes it.

One-change rule: Write the symptom, change exactly one variable, then repeat the same observation. Otherwise a successful result does not identify the cause.

Observed symptom	One controlled change	Result / inference

10 Controlled extension: encode three states

Only after the base circuit has three successful cycles, modify *software only* to display this repeating sequence:

1. red on, blue off for 500 ms;
2. both on for 500 ms;
3. red off, blue on for 500 ms;
4. both off for 500 ms.

Predict first, then implement. This isolates the changed variable: timing/state logic changes while hardware remains fixed.

State	Red command	Blue command	Observed correctly?	Evidence
1	on	off	☐	
2	on	on	☐	
3	off	on	☐	
4	off	off	☐	

Optional engineering extension: Replace one resistor with 680 Ω while power is disconnected. Predict and compare brightness and calculated current. Restore the original part afterward. Never replace it with a

jumper.

11 Assessment

1. Draw arrows on the schematic showing conventional current when D4 is HIGH. Why is there no intended current in the D7 branch when D7 is LOW?

2. A green LED has $V_f = 2.2\,\text{V}$. Estimate current with a $330\,\Omega$ resistor and a 5 V HIGH. Show units.

3. Explain why the two resistors cannot be replaced by one resistor in the shared ground path if predictable independent brightness is required.

4. What evidence distinguishes "the blue LED is reversed" from "D7 is never commanded HIGH"?

5. State the safe procedure for moving a jumper and explain why software `off()` alone is not a substitute.

6. What must `shutdown()` guarantee, and why must a destructor be `noexcept` in both exception-disabled and exception-enabled programs?

11.1 Claim–Evidence–Reasoning

Question: Can the Mega control the two external LED branches independently?

Claim (one precise sentence):

Evidence (cite at least three cycles and measured pin voltages; do not write only "it worked"):

Reasoning (connect output voltage, current path, resistor, LED polarity, and observations to the claim):

12 Completion check

- ☐ Predictions were recorded before construction.
- ☐ Every wiring change occurred with all power disconnected.
- ☐ Each LED had its own verified series resistor.
- ☐ HIGH/LOW voltages and three behavioral cycles were recorded.
- ☐ A controlled extension changed only one planned variable set.
- ☐ Troubleshooting, assessment, and CER are complete.
- ☐ Final state is documented: ☐ safely powered for demonstration or ☐ all power disconnected and parts left de-energized.

Core habit: predict → inspect de-energized → energize → measure repeatedly → explain from evidence.